



**DG INTERNS HUB**

# **OWASP TOP 10**

## **Study Notes on Common Web Application Vulnerabilities**

*SQL Injection | Cross-Site Scripting (XSS) | Broken Authentication | Security Misconfiguration*

Prepared as part of Cybersecurity Internship

Week 6: Web Security & Vulnerability Testing (VAPT Basics)

**DG Interns Hub**

By: Avisha Masih

Cybersecurity Intern | SIN: DG/AUGUST/CYBER/098

September 2026

# Introduction

---

The OWASP Top 10 is a globally recognised, regularly updated document published by the Open Web Application Security Project (OWASP) that lists the most critical security risks to web applications. It is used by developers, security testers, and organisations worldwide as the starting point for building secure software and for prioritising remediation during Vulnerability Assessment and Penetration Testing (VAPT). These notes cover four vulnerabilities from the OWASP Top 10 that were studied and practically tested during this internship task using DVWA (Damn Vulnerable Web Application), Burp Suite, and OWASP ZAP: SQL Injection, Cross-Site Scripting (XSS), Broken Authentication, and Security Misconfiguration.

## 1. SQL Injection (SQLi)

---

### 1.1 What is SQL Injection?

SQL Injection is a code-injection vulnerability that occurs when untrusted user input is inserted directly into a SQL query without proper validation, sanitisation, or the use of parameterised statements. Because the database cannot distinguish between the intended query logic and the attacker-supplied data, a carefully crafted input can change the structure and meaning of the original query. This allows an attacker to read, modify, or delete data they should never be able to access, and in severe cases to bypass authentication entirely or execute administrative operations on the database server.

### 1.2 How the Attack Works

Consider a typical login or search feature where the backend builds a query by directly concatenating user input, such as:

```
SELECT * FROM users WHERE id = '$id';
```

If the application does not sanitise the value of \$id, an attacker can enter a value like the one below instead of a normal ID:

```
1' OR '1'='1
```

Once substituted into the query, it becomes:

```
SELECT * FROM users WHERE id = '1' OR '1'='1';
```

Since the condition '1'='1' is always true, the WHERE clause is effectively neutralised and the query returns every row in the users table instead of a single record. This is exactly the behaviour that was reproduced during hands-on testing on the DVWA "SQL Injection" module (Security Level: Low), where entering 1' OR '1'='1 in the User ID field returned the details of all five users (admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith) in one response, and a UNION-based payload was used to retrieve usernames and password hashes directly from the users table.

### 1.3 Types of SQL Injection

- **In-band (Classic) SQLi:** The attacker uses the same channel to inject the payload and retrieve results, e.g. Union-based and Error-based SQLi.
- **Blind SQLi:** The application does not return database data directly; the attacker infers information from the application's behaviour, using Boolean-based (true/false responses) or Time-based (response delay) techniques.
- **Out-of-band SQLi:** The attacker retrieves data through a different channel, such as DNS or HTTP requests triggered by the database server, used when in-band techniques are not feasible.

### 1.4 Impact

- Unauthorised access to sensitive data such as usernames, passwords, and personal or financial records.
- Authentication bypass, allowing an attacker to log in without valid credentials.
- Data modification or deletion, leading to loss of data integrity.
- In severe cases, full compromise of the underlying database server and, depending on configuration, the host system itself.

### 1.5 Mitigation / Prevention

- Use parameterised queries (prepared statements) or an ORM instead of building SQL strings by concatenation.
- Apply strict server-side input validation (whitelisting expected formats such as numeric IDs).
- Enforce the principle of least privilege for database accounts used by the application.
- Use stored procedures carefully, ensuring they do not themselves concatenate unsanitised input.
- Deploy a Web Application Firewall (WAF) as a defence-in-depth measure, and disable detailed database error messages in production.

## 2. Cross-Site Scripting (XSS)

---

### 2.1 What is XSS?

Cross-Site Scripting is a client-side injection vulnerability where an attacker injects malicious scripts (typically JavaScript) into web pages that are then viewed by other users. Because the browser trusts content served by the website, the injected script executes in the context of that site, giving the attacker access to cookies, session tokens, and other sensitive data available to the page, or allowing the page's appearance and behaviour to be altered without the victim's knowledge.

### 2.2 Types of XSS

- **Reflected XSS:** The malicious script is part of the request (e.g. a URL parameter or form field) and is immediately reflected back in the server's response without being stored. The victim must be tricked into clicking a crafted link. During testing, the DVWA "XSS (Reflected)" module was used with the payload `<script>alert('XSS')</script>` entered in the name field, which produced an immediate JavaScript alert box confirming the script executed in the browser.
- **Stored (Persistent) XSS:** The malicious script is permanently saved on the target server, for example in a database, comment field, or guestbook entry, and is served to every user who later views that page — making it more dangerous than reflected XSS. This was demonstrated on the DVWA "XSS (Stored)" guestbook module, where the payload `<script>alert('Stored XSS')</script>` was submitted as a message; the alert triggered immediately and re-triggered on page reload, confirming the script was stored in the database rather than just reflected.
- **DOM-based XSS:** The vulnerability exists in client-side JavaScript itself, where untrusted data is written into the Document Object Model (DOM) without sanitisation, causing the script to execute purely on the client side without the payload ever being sent to or reflected by the server.

### 2.3 Example Payload

```
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS')>
```

In a real attack, instead of a harmless alert box, the payload would typically read `document.cookie` and send it to an attacker-controlled server, enabling session hijacking.

### 2.4 Impact

- Theft of session cookies, leading to session hijacking and account takeover.
- Defacement of the web page or redirection of users to malicious sites.
- Keystroke logging or credential phishing via injected fake login forms.
- Spreading malware or performing actions on behalf of the victim (e.g. via CSRF combined with XSS).

## 2.5 Mitigation / Prevention

- Perform context-aware output encoding/escaping of all user-supplied data before rendering it in HTML, JavaScript, or URL contexts.
- Validate and sanitise input on the server side; never rely on client-side validation alone.
- Implement a strong Content-Security-Policy (CSP) header to restrict which scripts can execute on the page.
- Set the HttpOnly and Secure flags on cookies so they cannot be accessed via JavaScript or sent over plain HTTP.
- Use modern frameworks (React, Angular, etc.) that auto-escape output by default, and avoid dangerous sinks such as `innerHTML` with untrusted data.

## 3. Broken Authentication

---

### 3.1 What is Broken Authentication?

Broken Authentication refers to a category of flaws in the mechanisms that verify a user's identity and manage their session once logged in. When authentication and session-management functions are implemented incorrectly, attackers can compromise passwords, session tokens, or keys, or exploit other implementation flaws to assume other users' identities, temporarily or permanently, without ever needing to guess a valid password through brute force alone.

### 3.2 Common Weaknesses

- **Weak or default credentials:** Applications that ship with default logins (e.g. admin/password, as used by DVWA itself for lab purposes) or that allow users to set very weak passwords.
- **No protection against brute-force / credential stuffing:** Absence of account lockout, rate-limiting, or CAPTCHA allows attackers to try thousands of password combinations automatically.
- **Insecure session management:** Predictable session IDs, session tokens that do not expire, or session identifiers exposed in the URL rather than secure cookies (related to the DVWA "Weak Session IDs" module).
- **Session fixation:** The application does not generate a new session ID after a successful login, allowing an attacker to "fix" a known session ID on a victim and then hijack the authenticated session.
- **Credentials sent over unencrypted channels:** Login forms submitted over plain HTTP expose usernames and passwords to network eavesdroppers.
- **Missing multi-factor authentication (MFA):** Reliance on a single factor (password only) for accessing sensitive functionality.

### 3.3 Example Scenario

During testing, DVWA's "Brute Force" module illustrates this weakness directly: because there is no lockout policy or CAPTCHA at low security, an attacker can repeatedly submit login attempts (manually or with a tool such as Burp Suite's Intruder) until the correct username/password combination is found. In a real application this same absence of rate-limiting is what makes credential-stuffing attacks against reused passwords practical at scale.

### 3.4 Impact

- Unauthorised account takeover, exposing personal data and any privileges tied to that account.
- Escalation to administrative access if an administrator account is compromised.
- Reputational and financial damage for the organisation if a breach traces back to weak authentication controls.

### 3.5 Mitigation / Prevention

- Enforce strong password policies and check new passwords against known-breached password lists.
- Implement account lockout, rate-limiting, and CAPTCHA after a small number of failed login attempts.

- Issue a new, random, high-entropy session ID on every login and invalidate the old one (preventing session fixation).
- Transmit credentials and session cookies only over HTTPS, with Secure, HttpOnly, and SameSite cookie attributes.
- Offer and, where possible, enforce multi-factor authentication (MFA) for sensitive accounts.
- Set reasonable session expiry/idle-timeout periods and allow users to view and revoke active sessions.

## 4. Security Misconfiguration

---

### 4.1 What is Security Misconfiguration?

Security Misconfiguration occurs when security settings are not defined, implemented, or maintained correctly at any layer of the application stack — the web server, application server, database, framework, or cloud platform. Unlike a coding flaw, misconfiguration is usually an operational gap: a secure feature exists but has been left disabled, left at an insecure default, or simply forgotten about. Because it can occur at so many layers, it is one of the most commonly found issues in real-world security assessments, and several instances of it were identified automatically during the OWASP ZAP scan performed in this task.

### 4.2 Common Examples

- **Missing security headers:** The OWASP ZAP scan against the DVWA instance flagged a missing Content-Security-Policy (CSP) header, a missing X-Frame-Options / anti-clickjacking header, and a missing X-Content-Type-Options header — each of which removes a browser-level layer of defence against XSS, clickjacking, and MIME-sniffing attacks respectively.
- **Information leakage:** The scan also detected that the server discloses its exact software version via the Server HTTP response header, and that a page reveals internal details through an "In Page Banner Information Leak" — information that helps an attacker fingerprint the stack and search for known, version-specific exploits.
- **Insecure cookie configuration:** A "Cookie without SameSite Attribute" finding means the session cookie can be sent along with cross-site requests, increasing exposure to CSRF-style attacks.
- **Verbose error messages:** Stack traces or raw database errors shown to end users can reveal file paths, query structure, or framework versions.
- **Unnecessary features enabled:** Default accounts, sample applications, directory listing, or unused ports/services left active on production servers.
- **Outdated software:** Running unpatched versions of the web server, framework, or CMS with known, publicly documented vulnerabilities.

### 4.3 Impact

- Provides attackers with reconnaissance information that speeds up further, more damaging attacks.
- Removes browser-enforced protections that would otherwise reduce the impact of XSS and clickjacking.
- Can expose entire systems if default credentials or unnecessary administrative interfaces are left accessible.

### 4.4 Mitigation / Prevention

- Harden all server and application configurations using a documented, repeatable process ("secure by default" baseline), and re-apply it consistently across dev, test, and production environments.
- Explicitly set security headers: Content-Security-Policy, X-Frame-Options (or the frame-ancestors CSP directive), X-Content-Type-Options: nosniff, and Strict-Transport-Security.
- Set the SameSite attribute on cookies and mark session cookies Secure and HttpOnly.
- Remove or disable default accounts, sample content, and unused services/ports; suppress the Server header or reduce the detail it reveals.

- Disable verbose/debug error pages in production and log detailed errors server-side only.
- Keep all software components patched and run periodic automated scans (such as with OWASP ZAP) to catch configuration drift.

## Summary Table

---

| Vulnerability              | Root Cause                                                              |
|----------------------------|-------------------------------------------------------------------------|
| SQL Injection              | Unsanitised user input concatenated directly into SQL queries           |
| Cross-Site Scripting (XSS) | Unsanitised/unencoded user input rendered back into HTML or JavaScript  |
| Broken Authentication      | Weak login, session, and password-management controls                   |
| Security Misconfiguration  | Insecure defaults, missing headers, or unpatched/unhardened environment |

*These four vulnerabilities were practically observed and confirmed during this Cybersecurity Internship task at DG Interns Hub using DVWA as the intentionally vulnerable target application, with Burp Suite used for manual request interception and exploitation, and OWASP ZAP used for automated vulnerability scanning — as documented in the accompanying VAPT project report.*